

C 语言作业题及源码答案

北京理工大学 C 语言作业题

2010/3/1

栾海鹏

QQ: 791808990

Email: ttcc.lhp@gmail.com



关于本书

本书作者栾海鹏，是山东大学威海分校的计算机科学与技术专业的 08 级学生，大二的专业名称叫电气信息类，可惜名不副实，学的知识与电气方面似乎没多大关系，就连计算机专业最基本的 C 语言必修课在大一就没开。

大二，栾海鹏作为山东大学威海分校计算机科学与技术专业的学生被派去北京理工大学作为交流生，在北理学习一年。北理计算机类专业对于编程基本功是很重视的，大一上学期就学习了很完整的 C 语言，大一下学期他们专门开设编程课，是必修课。大二上学期他们便开设了数据结构的课程。对于我们交流生来说，大一根本没有 C 语言基础。那时候上数据结构课的时候还没什么感觉，就是课后的编程作业，真的让人很头疼，每周都会出两道三个编程作业，是在网络教室上做的，在线提交，立刻在网上就输出结果，测试用例好多呢，一个测试用例通过不了，这个题就没做对。每完成一个题目都要花好长时间的，至少要一下午的时间。

不过还好，幸亏在大一的暑假里我自学了 C 语言，很努着力把数据结构的十三个编程题 AC（all correct）了一半。

本书包含了大一初学 C 语言的作业题，不设计复杂的数据结构。题目是北理网络教室里的题目，答案是栾海鹏自己编写的（作业题嘛）。

由于水平有限，难免出现错误，读者发现错误，欢迎指证批评，共同进步。

邮箱 ttcc.lhp@gmail.com QQ : 791808990

栾海鹏

二〇一〇年三月十八日

目录

1 温度转换.....	5
2 计算圆柱的侧面积及体积.....	6
3 计算时钟的夹角.....	7
4 判断三角形的形状.....	8
5 求解一元二次方程.....	8
6 计算 SUM 的值.....	10
7 贪吃的猴子.....	11
8 求最后 3 位数值.....	12
9 分数的四则运算.....	13
10 黑色星期五.....	16
11 百马百担.....	18
12 零钱换整钱.....	19
13 找出最大素数.....	20
14 空心的倒三角型.....	21
15 空心字符菱形.....	22
16 邮票组合（选作）.....	23
17 谁能出线.....	25
18 等值数列段.....	27
19 大家一起做游戏.....	29
20 猜数字（选作）.....	31
21 小蜜蜂（选作）.....	32
22 数制转换（选作）.....	33
23 删除重复字符.....	35
24 单词排序.....	37
25 大数分解.....	38
26 铺地板（选作）.....	40
27 编码问题（选作）.....	42
28 洗牌（选作）.....	44
29 扫雷（选做）.....	45

30 杀鸡用牛刀——要用递归啊！	48
31 求最大公约数——递归	49
32 求序列之和——递归	50
33 回文字符串——递归	51
34 科学记数法	52
35 逆波兰算术表达式（选作）	54
36 身份证的奥秘（选做）	57
37 合并字符串	62
38 折半插入排序	64
39 拱猪计分（选作）	67
40 合并果子（选作）	77
41 安全的密码（选做）	80
2. 集合排序	83
3. 二叉排序树	89
42 学生成绩排序	93
43 缩写形式的展开（选作）	96
44 链表排序	100
45 子串反序--递归	104

1 温度转换

背景：

经常出国旅行的驴友都知道，需要时时了解当地的气温状况，但不少国家采用了不同的温度计量单位：有些使用华氏温度标准（F），有些使用摄氏温度（C）。现在，请你根据温度转换公式设计一个温度转换程序，可以进行温度转换。如果输入摄氏温度，显示转换的华氏温度；如果输入华氏温度，显示转换的摄氏温度。

温度转换的公式为： $F=(C \times 9 / 5) + 32$ ； $C=(F - 32) \times 5 / 9$ ；式中 F--华氏温度，C--摄氏温度。

输入：

第一个数据（1 或 2）确定转换的类型：华氏转摄氏（1）；摄氏转华氏（2）；
第二个数据是相应需要转换的温度值。

输出：

相应的转换后的温度值（保留小数点后 2 位）。

```
#include<stdio.h>
void main()
{
    int i;
    float j;
    float a;
    scanf("%d%f",&i,&j);
    if(i==1)
    {
        a=(j-32)*5.0/9.0;
        printf("The Centigrade is %.2f\n",a);
    }
    else if(i==2)
    {
        a=(j*9/5.0)+32.0;
        printf("The Fahrenheit is %.2f\n",a);
    }
}
```

2 计算圆柱的侧面积及体积

如果已知圆柱的底面半径 r ，以及高 h ，则可计算出圆柱的侧面积 $s=2\pi rh$ ，体积 $v=\pi r^2 h$ 。其中 $\pi=3.1415926$

输入

第一行输入圆柱的底面半径 r

第二行输入圆柱的高 h

输出

$s=<$ 圆柱的侧面积 $>,v=<$ 圆柱的体积 $>$

要求

1. 所有变量都定义为双精度类型
2. 结果精确到小数点后两位

```
#include<stdio.h>
#define PI 3.1415926
void main()
{
    float r,h;
    float s,v;
    scanf("%f",&r);
    scanf("%f",&h);
    s=2*PI*r*h;
    v=PI*r*r*h;
    printf("s=%.2f, v=%.2f\n", s, v);
}
```

3 计算时钟的夹角

背景：

钟面上的时针和分针之间的夹角总是在 0° ~ 180° 之间（包括 0° 和 180° ）。举例来说，在十二点的时候两针之间的夹角为 0° ，而在六点的时候夹角为 180° ，在三点的时候为 90° 。本题要解决的是计算 12:00 到 11:59 之间任意一个时间的夹角。

输入：

每组测试数据包含两个数字：第一个数字代表小时（大于 0 小于等于 12），第二个数字代表分（在区间 $[0, 59]$ 上）。

输出：

对应每组测试数据，用常用格式显示时间以及这个时候时针和分针间的最小夹角，精确到小数点后一位。输出格式如下所示。

```
#include<stdio.h>
#include<math.h>
#define PI 3.1415926
void main()
{
    int h, hh, m;
    float degree, M_degree, H_degree;
    scanf("%d%d", &h, &m);
    M_degree=6*m;
    hh=h;
    if(h==12) h=0;
    H_degree=(h+m/60.0)*30.0;
    degree=H_degree-M_degree;
    degree=fabs(degree);
    if(h==0) h=12;
    if(m<10)
        printf("At %d:0%d the angle is %.1f degrees.\n", hh, m, degree);
    else printf("At %d:%2d the angle is %.1f degrees.\n", hh, m, degree);
}
```

4 判断三角形的形状

要求：

输入三角型的三条边，判断三角形的形状。假设输入的三边边长均 >0 。

输入：

三角型的 3 条边的长度（int 型）。

输出：

等边三角形：equilateral triangle

等腰三角形：isocetes triangle

不构成三角形：non-triangle

一般三角形：triangle

```
#include<stdio.h>

void main() {
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    if(a+b<c || b+c<a || c+a<b)
        printf("non-triangle.\n");
    else if(a==b&&b==c)
        printf("equilateral triangle.\n");
    else if(a==b || b==c || a==c)
        printf("isocetes triangle.\n");
    else printf("triangle.\n");
}
```

5 求解一元二次方程

成绩: 15 / 折扣: 0.8

初中的时候我们就会求解一元二次方程了，下面让我们来教计算机如何求解。输入 a, b, c ，就一元二次方程 $ax^2+bx+c=0$ 的根。

输入：

假设 a, b, c 均 int。

输出：

要求输出的根为 `double` 型，保留 6 位小数。

```
#include<stdio.h>
#include<math.h>
void main()
{
    int a, b, c;
    float d, delta, x1, x2, x11, x22;
    scanf("%d%d%d", &a, &b, &c);
    d=b*b-4*a*c;
    if(a==0&&b==0)printf("Input error!\n");
    else if(a==0)
    {
        x1=(0-(float)c)/b;
        printf("x=%6f\n", x1);
    }
    else if(d<0)
    {
        d=0-d;
        delta=sqrt(d);
        if(b==0)
        {
            x2=(0-b-delta)/(2*a);
            x1=(-b+delta)/(2*a);
            printf("x1=%6fi\n", x1);
            printf("x2=%6fi\n", x2);
        }
        else
        {
            x11=-(float)b/(2.0*a);
            x1=(0+delta)/(2*a);
            x2=(0-delta)/(2*a);
            printf("x1=%6f+%6fi\n", x11, x1);
            printf("x2=%6f%6fi\n", x11, x2);
        }
    }
    else
    {
        d=b*b-4*a*c;
        delta=sqrt(d);
        if(d>0)
        {
            x2=(0-b-delta)/(2*a);
```

```
        x1=(-b+delta)/(2*a);
        printf("x1=%6f\n", x1);
        printf("x2=%6f\n", x2);
    }
    else if(d==0)
    {
        x1=(-b+delta)/(2*a);
        printf("x1=x2=%6f\n", x1);
    }
}
}
```

6 计算 SUM 的值

已知公式： $SUM = 1 + 1/2 + 1/3 + 1/4 + \dots + 1/n$

输入：

n

输出：

表达式 sum 的值。结果保留 6 位小数。

```
#include<stdio.h>

void main()
{
    int n;
    double sum=0;
    scanf("%d",&n);
    for(int i=1;i<=n;i++) sum=sum+1.0/i;
    printf("sum=%6lf\n", sum);
}
```

7 贪吃的猴子

有一只猴子，第一天摘了若干个桃子，当即吃了一半，但还觉得不过瘾，就又多吃了一个。第2天早上又将剩下的桃子吃掉一半，还是觉得不过瘾，就又多吃了两个。以后每天早上都吃了前一天剩下的一半加天数个（例如，第5天吃了前一天剩下的一般加5个）。到第n天早上再想吃的时候，就只剩下一个桃子了。

输入：

天数 n

输出：

第一天的桃子个数

```
#include<stdio.h>
int main()
{
    int n, x=1;
    scanf("%d", &n);
    for(n; n>0; n--)
        x=2*(n-1)+2*x;
    x=x/2;
    if(x>1)
        printf("The monkey got %d peachs in first day.\n", x);
    else if(x==1) printf("The monkey got %d peach in first day.\n", x);
    return 0;
}
```

8 求最后 3 位数值

中学时我们就会求一个数的 n 次方了，但计算机不会，请你来编写一个程序吧。由于计算机比较“笨”，所以我们编写程序的功能不用太强，只要能算出 a 的 n 次方的最后 3 位数就可以了。

输入：

a 和 n 的值。假设 $a \leq 150$ 。

输出：

求 a 的 n 次方的最后 3 位数。

```
#include<stdio.h>
int main()
{
    int m,n;
    int S=1;
    scanf("%d%d",&m,&n);
    if(n!=0)
    {
        for(int i=0;i<n;i++)
        {
            S=S*m;
            S=S%1000;
        }
        printf("The last 3 numbers is %.3d.\n",S);
    }
    else
        printf("The last 3 numbers is 1.\n",S);
    return 0;
}
```

9 分数的四则运算

在小学时我们就学习了分数的四则运算，即对两个分数进行加、减、乘、除等运算，现在我们尝试下用 C 语言来实现。

输入：

分数 1 操作符 分数 2

输出：

计算结果

要求：

计算结果使用分数表示，并且为最简化。例如结果为 2/6，则被简化为 1/3

```
#include<stdio.h>

int Gcd(int a, int b) //求最大公约数，以便对结果进行约分
{
    while(a!=0&&b!=0)
    {
        if(a>b) a=a%b;
        else if(a<b) b=b%a;
    }
    return (a>b?a:b);
}

void plus(int a, int b, int d, int e) //计算加法
{
    int fenzi, fenmu;
    int g;
    fenmu=b*e;
    fenzi=a*e+b*d;
    g=Gcd(fenmu, fenzi);
    fenmu=fenmu/g; //对分母约分
    fenzi=fenzi/g; //对分子约分
    if(fenzi==0)
        printf("%d/%d + %d/%d = %d\n", a, b, d, e, fenzi);
    else
        printf("%d/%d + %d/%d = %d/%d\n", a, b, d, e, fenzi, fenmu);
}

void minus(int a, int b, int d, int e) //计算减法
```

```
{
    int fenzi, fenmu;
    int g;
    fenmu=b*e;
    fenzi=a*e-b*d;
    if(fenzi>=0&&fenmu>=0)
    {
        g=Gcd(fenmu, fenzi);
        fenmu=fenmu/g; //对分母约分
        fenzi=fenzi/g; //对分子约分
    }
    if(fenzi==0)
        printf("%d/%d - %d/%d = %d\n", a, b, d, e, fenzi);
    else
        printf("%d/%d - %d/%d = %d/%d\n", a, b, d, e, fenzi, fenmu);
}

void multiply(int a, int b, int d, int e) //乘法
{
    int fenzi, fenmu;
    int g;
    fenzi=a*d;
    fenmu=b*e;
    g=Gcd(fenmu, fenzi);
    fenmu=fenmu/g; //对分母约分
    fenzi=fenzi/g; //对分子约分
    if(fenzi==0)
        printf("%d/%d * %d/%d = %d\n", a, b, d, e, fenzi);
    else if(fenmu==1)
        printf("%d/%d * %d/%d = %d\n", a, b, d, e, fenzi);
    else
        printf("%d/%d * %d/%d = %d/%d\n", a, b, d, e, fenzi, fenmu);
}

void divide(int a, int b, int d, int e) //除法
{
    int fenzi, fenmu;
    int g;
    fenzi= a * e;
    fenmu= b * d;
    g=Gcd(fenmu, fenzi);
    fenmu=fenmu/g; //对分母约分
    fenzi=fenzi/g; //对分子约分
    if(fenzi==0)
        printf("%d/%d / %d/%d = %d\n", a, b, d, e, fenzi);
```

```
        else
            printf("%d/%d / %d/%d = %d/%d\n", a, b, d, e, fenzi, fenmu);
    }

    void main()
    {
        int a, b, d, e;
        char c;
        scanf("%d/%d %c %d/%d", &a, &b, &c, &d, &e);
        switch(c)
        {
            case '+': plus(a, b, d, e);break;
            case '-': minus(a, b, d, e);break;
            case '*': multiply(a, b, d, e);break;
            case '/': divide(a, b, d, e);break;
        }
    }
}
```

10 黑色星期五

成绩: 15 / 折扣: 0.8

在西方，星期五和数字 13 都代表着坏运气，两个不幸的个体最后结合成超级不幸的一天。所以，不管哪个月的十三日又恰逢星期五就叫“黑色星期五”。

输入：

年份

输出：

判断该年是否包含黑色星期五，如包含，给出具体日期

```
#include<stdio.h>
typedef struct
{
    int month;
    int day;
}Data;

int main()
{
    Data data[10];
    int a[13]={0, 12, 43, 71, 102, 132, 163, 193, 224, 255, 285, 316, 346};
    int b[13]={0, 12, 43, 72, 103, 133, 164, 194, 225, 256, 286, 317, 347}; //闰年时
    int n=0, i, year;
    scanf("%d", &year);
    if(((year%4==0)&&(year%100!=0)) || (year%400==0)) //闰年时
    {
        for(i=1; i<=12; i++)
        {
            if((year*365+(year-1)/4-year/100+year/400+b[i])%7==5)
            {
                data[n].month=i;
                data[n].day=13;
                n++;
            }
        }
    }
    else //平年
    {
        for(i=1; i<=12; i++)
```



```
{
    if((year*365+year/4-year/100+year/400+a[i])%7==5)
    {
        data[n].month=i;
        data[n].day=13;
        n++;
    }
}

if(n==1)
{
    printf("There is %d Black Friday in year %d. \nIt is:\n", n, year);
    printf("%d/%d/%d\n", year, data[0].month, data[0].day);
}
else
{
    printf("There are %d Black Fridays in year %d. \nThey are:\n", n, year);
    for(i=0; i<n; i++) printf("%d/%d/%d\n", year, data[i].month, data[i].day);
}

return 0;
}
```

11 百马百担

有 100 匹马，驮 100 担货，其中大马驮 3 担，中马驮 2 担，两匹小马驮 1 担

输入：

无

输出：

大、中、小马的个数，用逗号分隔。例如：2,30,68

说明：

大、中、小马都必须有，结果中要列出所有的组合可能每个结果占一行
因为有多组结果，结果的排序按照大马的个数从少到多

```
#include<stdio.h>
int main()
{
    int i, j, k;
    //有匹马，驮担货，其中大马驮担，中马驮担，两匹小马驮担
    //输出：大、中、小马的个数，用逗号分隔。例如：,30,68
    for(i=1;i<=100;i++)
    {
        for(j=1;j<=100-i;j++)
        {
            k=100-i-j;
            if((3*i+2*j+k/2.0)==100)
                printf("%d,%d,%d\n", i, j, k);
        }
    }
    return 0;
}
```

12 零钱换整钱

小明去银行存钱，拿了一堆硬币。已知 1 角的硬币厚度为 1.8mm，5 角的硬币厚 1.5mm，1 元的硬币为 2.0mm。小明将 1 角的硬币放成一摞，将 5 角的硬币放成一摞，将 1 元的硬币放成一摞，发现 3 摞硬币一样高。银行正好把这些硬币换成若干张面值为 10 元的纸币。问小明至少带了多少钱的硬币（元为单位），每种面值的硬币有多少个？

输入：

无

输出：

1 角的数量,5 角的数量,1 元的数量,总金额。（例如：20,30,40,50）

说明

在结果中只输出各个对应的数字即可

```
#include<stdio.h>
void main()
{
    int yijiao;
    int wujiao;
    int yiyuan;
    int sum;
    for(yiyuan=0;yiyuan<100;yiyuan++)
    {
        for(wujiao=0;wujiao<100;wujiao++)
        {
            for(yijiao=0;yijiao<100;yijiao++)
            {
                sum=yijiao+wujiao*5+yiyuan*10;

                if((sum>=100)&&(sum%100==0)&&(18*yijiao==15*wujiao)&&(15*wujiao==20*yiyuan))
                {
                    printf("%d, %d, %d, %d\n", yijiao, wujiao, yiyuan, sum/10);
                    break;
                }
            }
        }
    }
}
```

13 找出最大素数

素数是指一个只能被 1 和它本身整除的数，在数论中占有重要的研究地位，在当代密码学中也广泛应用。

输入：

取值范围

输出：

该范围内的最大素数

```
#include<stdio.h>
int sushu(int m)
{
    int i;
    for( i=2;i<m;i++)
    {
        if(m%i==0)break;
    }
    if(i==m)return 1;
    else return 0;
}
void main()
{
    int n;
    scanf("%d",&n);
    for(int i=n;i>1;i--)
    {
        if(sushu(i))
        {
            printf("The max prime number is %d.\n",i);
            break;
        }
    }
}
```

14 空心的倒三角型

背景：

请根据要求打印可空心倒三角形。

输入：

输入三角形的高度（ $h \geq 0$ ）。

输出：

打印相应的空心倒三角形。

```
#include<stdio.h>
int main()
{
    int h, j, k;
    scanf("%d", &h);
    for(j=0; j<(h*2-1); j++)printf("*");
    printf("\n"); //输出第一行
    for(int i=1; i<h; i++)
    {
        for(k=0; k<i; k++)printf(" ");
        printf("*");
        for(k=0; k<(h*2-3-i-i); k++)printf(" ");
        if(i!=h-1)printf("*");
        printf("\n");
    }
    return 0;
}
```

15 空心字符菱形

输入：

菱形起始字母和菱形的高度。

输出：

参看测试用例，打印空心的由字符组成的菱形。

测试输入

期待的输出

测试用例 1

1. C 4*

```
1.  C*
2.  D D*
3.  E  E*
4.  F  F*
5.  E  E*
6.  D D*
7.  C*
```

```
#include<stdio.h>

void main()
{
    char letter;
    int i, j, k, l, n;
    scanf("%c%d", &letter, &n);
    for(i=0; i<n; i++)
    {
        if(i==0)
        {for(j=0; j<n-1; j++)printf(" ");printf("%c", letter);}
        else if(i>0)
        {
            for(j=0; j<n-1-i; j++)printf(" ");
            printf("%c", letter+i);
            for(j=0; j<2*i-1; j++)printf(" ");
            printf("%c", letter+i);
        }
        printf("\n");
    }
    for(i=n-2; i>=0; i--)
    {
        if(i==0)
        {for(j=0; j<n-1; j++)printf(" ");printf("%c", letter);}
        else if(i>0)
        {
            for(j=0; j<n-1-i; j++)printf(" ");
            printf("%c", letter+i);
            for(j=0; j<2*i-1; j++)printf(" ");
            printf("%c", letter+i);
        }
    }
}
```

```
        printf("\n");  
    }  
}
```

16 邮票组合（选作）

背景：

我们寄信都要贴邮票，在邮局有一些小面值的邮票，通过这些小面值邮票中的一张或几张的组合，可以满足不同邮件的不同的邮资。

现在，邮局有 4 种不同面值的邮票。在每个信封上最多能贴 5 张邮票，面值可相同，可不同。

输入：

四种邮票的面值。

输出：

用这四种面值组成的邮资最大的从 1 开始的一个连续的区间。

说明：

如结果为 10，则表明使用 4 张邮票可组合出 1、2、3、4、5、6、7、8、9、10 这些邮资。

名词解释：

邮资：就是你寄东西需要花多少钱。

邮票面额：是由国家发行的具有固定价格的花纸片，被称为邮票。

如果你寄东西，邮局称了重量，告诉你要 240 分。这样你就要贴邮票了。如果现在邮局的邮票有面值为 80 分、50 分、20 分和 10 分的四种，你就可以采用不同的组合得到 240 的邮资，例如：采用 3 张 80 分的可以凑出 240 分；或者 24 张 10 分的凑起来 240 分也可以。显然不同邮票的组合都可以得到同一种邮资。

```
#include<stdio.h>  
  
int neng(int a[], int max)  
{  
    int i, j, k, m, n;  
    int sum;  
    for(i=0; i<5; i++)  
        for(j=0; j<5; j++)
```

```
        for(k=0;k<5;k++)
            for(m=0;m<5;m++)
                for(n=0;n<5;n++)
                {
                    sum=a[i]+a[j]+a[k]+a[m]+a[n];
                    if(sum==max) return 1;
                }

        return 0;
    }
void main()
{
    int a[5],i;
    int max=0;
    a[4]=0;
    scanf("%d%d%d%d", &a[0], &a[1], &a[2], &a[3]);
    while(1)
    {
        if(neng(a, max)) {
            max++;
        }
        else break;
    }
    printf("The max is %d. \n", max-1);
}
```


17 谁能出线

背景：

电视台举办“超级学生”才艺大赛，由于报名人数狂多，所以要先进行分组预赛。按规定，每 10 名学生为一个预赛小组，评委打出分数（0~100 分），各小组第一名可以进入下一轮；如果有多名学生得分相同都是第一名，则可同时进入下一轮。

输入：

按顺序给出一个小组 10 个人的最后得分（int）。

输出：

能够出线的学生序号（0~9）。

```
#include<stdio.h>
typedef struct
{
    int num;
    int score;
}student;
void sort(student A[], int N)
{
    int i, j;
    for(i=0; i<N; i++)
        for(j=0; j<N-1; j++)
        {
            if(A[j].score<A[j+1].score)
            {
                int temp;
                temp=A[j].score;
                A[j].score=A[j+1].score;
                A[j+1].score=temp;
                temp=A[j].num;
                A[j].num=A[j+1].num;
                A[j+1].num=temp;
            }
        }
}
void main()
{
    student stu[100];
```

```
int i;
for(i=0;i<10;i++)
{
    scanf("%d",&stu[i].score);
    stu[i].num=i;
}
sort(stu, 10);
for(i=0;i<10;i++)
{
    printf("%d\n", stu[i].num);
    if(stu[i].score!=stu[i+1].score)break;
}
}
```

18 等值数列段

如果一个数列中的某一段（至少有两个元素）的各元素值均相同，则称之为等值数列段。等值数列段中元素的个数叫做等值数列段的长度。

输入：

由 N 个元素组成的整数数列 A（其中 $N \leq 50$ ）

输出：

A 中长度最大的所有等值数列段的始末位置，如果没有等值数列段，则输出 No equal number list.

说明：

始末位置是指数组下标，即 0 表示第一个元素。

如果有多个同等长度的等值数列，只输出第一个等值数列的起始位置。

当在一个 LIST 中出现两个等长的连续串的时候，我们的答案应该是第一个等长串。

```
#include<stdio.h>

struct from_to
{
    int from;
    int to;
};

void main()
{
    int a[1000];
    int N;
    scanf("%d",&N);
    int i;
    for(i=0;i<N;i++)
    {
        scanf("%d",&a[i]);
    }
    int j;
    struct from_to b[1000];
    b[0].from=0;
    j=0;
    for(i=1;i<N;i++)
    {
        if(a[i]!=a[i-1])
        {
            b[j].to=i;
            j++;
        }
    }
}
```

```
        b[j].from=i;
    }
}
b[j].to=N;
int c[1000];
for(i=0;i<=j;i++)
{
    c[i]=b[i].to-b[i].from;
}
int max=c[0];
int k=0;
for(i=0;i<=j;i++)
{
    if(max<c[i])
    {
        max=c[i];
        k=i;
    }
}
if(b[k].to==1) printf("No equal number list.\n");
else
printf("The longest equal number list is from %d to %d.\n",b[k].from,b[k].to-1);
}
```

19 大家一起做游戏

幼儿园的小朋友们刚学习了如何数数，阿姨在下课时组织大家一起玩游戏。规则如下：所有的小朋友绕成一圈，顺序排号，从第一个小朋友开始报数，凡是报到固定数字（例如 5）的，都退出该游戏，直到只剩下一位小朋友游戏才中止。

每个小朋友都希望自己能有更多的练习数数的机会，所以都希望成为最终被留下的那位。

现在，请大家帮小朋友们计算一下，在第一次排号的时候排到第几位才能成为最终被留下的小朋友。

输入：

小朋友的个数（ ≤ 50 ） 要被练习的数字

输出：

最终被留下的小朋友的序号

说明：

如“要被练习的数字”是 5，则每次数到 5 的同学要退出该游戏

```
#include<stdio.h>
#include<malloc.h>
typedef struct node
{
    int num;
    struct node * next;
}Node;

void main()
{
    Node head;
    head.num=1;
    int n, s;
    int i;
    scanf("%d%d", &n, &s);
    Node * p, *t;
    p=&head;
    for(i=2; i<=n; i++)
    {
        t=(Node *) malloc(sizeof(Node));
        t->num=i;
        p->next=t;
        p=t;
    }
```

```
p->next=&head;
int leftNum=n;
p=&head;
while(leftNum>1)
{
    for(i=1;i<=s;i++)
    {

        if(i==s-1)
        {
            p->next=p->next->next;
            leftNum--;
        }
        else
            p=p->next;
    }
}
printf("The left child is NO %d.\n",p->num);
}
```

20 猜数字（选作）

成绩: 5 / 折扣: 0.8

有如下一组数字，其中每个数字都在 1 ~ 63 之间，

1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63
2 3 6 7 10 11 14 15 18 19 22 23 26 27 30 31 34 35 38 39 42 43 46 47 50 51 54 55 58 59 62 63
4 5 6 7 12 13 14 15 20 21 22 23 28 29 30 31 36 37 38 39 44 45 46 47 52 53 54 55 60 61 62 63
8 9 10 11 12 13 14 15 24 25 26 27 28 29 30 31 40 41 42 43 44 45 46 47 56 57 58 59 60 61 62 63
16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63
32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63

现在需要编写一个程序，当从键盘输入某数字在且仅在哪几行出现了，该程序可以“猜”出这个数字是几。

输入：

程序的输入是一串以空白符分开的数字，当输入 0 时表示输入结束。

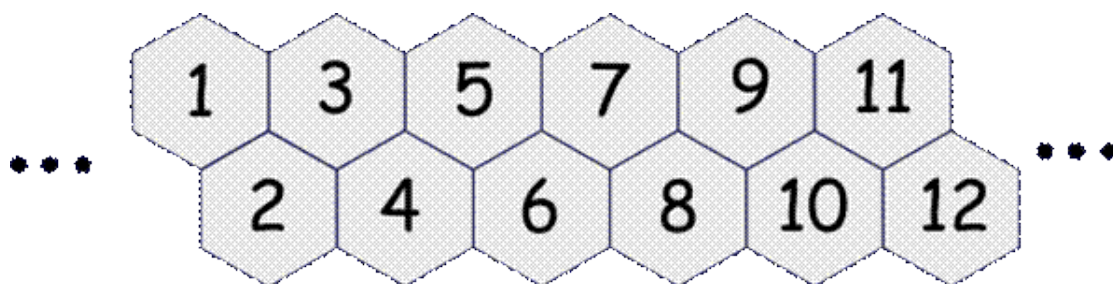
输出：

输出猜到的数字。

```
#include <stdio.h>
#include <math.h>
int main()
{
    int i, n=0;
    while(1)
    {
        scanf("%d", &i);
        if(i==0) break;
        n+=pow(2, i-1);
    }
    printf("%d\n", n);
    return 0;
}
```

21 小蜜蜂（选作）

一只小蜜蜂在如下图所示的蜂窝上爬行。它爬行时，只能从一个格爬到相邻的大号格子中。例如，从 1 号格子可以爬到 2 号或者 3 号格子，从 2 号则可以爬到 3 号或者 4 号格子。



请问从一个格子 a 爬到一个格子 b 一共有多少种可行的路线。

输入：

分别是起始点 a 和终止点 b 的编号。（a 和 b 在 1~100 之间，且 $a < b$ 。）

输出：

方案数量。

```
#include<stdio.h>
void main()
{
    int i;    long double a, b, n, un1, un2, un;
    scanf("%lld %lld", &a, &b);
    n=b-a+1;
    if(n==1)printf ("0\n");
    un = un2 = 1;
    if(n==2)    un=1;
    for ( i=3; i<=n; i++)
    {
        un1 = un2;
        un2 = un;
        un = un1 + un2;
    }
    printf ("%0.011f\n", un);
}
```


22 数制转换（选作）

我们通常使用的都是十进制的数字，但其实在生活中也经常会使用其它进制。

这个题目会给你两个不同的数字，它们不属于同一进制，要求你计算出当它们分别处于何种进制之中时，两个数字相等。譬如 12 和 5，在十进制下它们是不等的，但若 12 使用 3 进制而 5 使用六进制或十进制时，它们的值就是相等的。因此只要选择合适的进制，12 和 5 就可以是相等的。

程序的输入是两个数字 M 和 N（其十进制的值不超过 1000000000），它们的进制在 2~36 之间。对于十以下的数字，用 0~9 表示，而十以上的数字，则使用大写的 A~Z 表示。

求出分别在 2~36 哪种进制下 M 和 N 相等。若相等则输出相应的进制，若不等则输出错误信息。信息的格式见测试用例。

```
#include<stdio.h>
#include<string.h>
#include <math.h>
// a[]={A B C D}
int transform(char c)
{
    if(c<='9' &&c>='0')
        return (int)(c-48); //0的ASCII码为
    else if(c<='Z' &&c>='A')
        return (int)(c-65+10); //A的ASCII码为
    else if (c<='z' &&c>='a')
        return (int)(c-97+10); //a的ASCII码为
}

int max(int a[], int n)
{
    int max;
    max=a[0];
    for(int i=1; i<n; i++)
    {
        if(max<a[i]) max=a[i];
    }
    return max;
}

void compare(char a[], int i, char b[], int j, int * oo)
{
    int a_length, b_length;
    a_length=strlen(a);
    b_length=strlen(b);
```

```
//a[0]*pow(i, a_length);
int a_num[200], b_num[200];
int m, n;
for(m=0; m<a_length; m++)
    a_num[m]=transform(a[m]);
for(n=0; n<b_length; n++)
    b_num[n]=transform(b[n]);
int a_ten=0, b_ten=0;
for(m=0; m<a_length; m++)
{
    //0 1 2 3 A B C D
    a_ten=a_ten+a_num[m]*pow(i, a_length-m-1);
}
for(n=0; n<b_length; n++)
{
    //0 1 2 3 A B C D
    b_ten=b_ten+b_num[n]*pow(j, b_length-n-1);
}

if(a_ten==b_ten)
{
    if(j==max(b_num, b_length)+1)
    {
        printf("%s (base %d) = %s (base %d)\n", a, i, b, j);
        *oo=1;
    }
}
}

void main()
{
    char a[200], b[200];
    int i, j, oo=0;
    scanf("%s%s", a, b);

    for(i=2; i<=36; i++)
        for(j=2; j<=36; j++)
        {
            compare(a, i, b, j, &oo);
        }
    if(oo==0)
        printf("%s is not equal to %s in any base 2..36\n", a, b);
}
```

23 删除重复字符

背景：

输入一个长度不超过 100 的字符串，删除串中的重复字符。

输入：

输入要检查的字符串，长度不超过 100 个字符。例如：abacaeedabcdcd。

输出：

删除重复字符后的字符串。例如：abcded。

```
#include<stdio.h>
#include<string.h>
int already_saved(char lastchar, char newstr[], int j)
{
    int i=0;
    while(i<j)
    {
        if(lastchar==newstr[i])return 1;
        i++;
    }
    return 0;
}
void main()
{
    char str[100];
    char newstr[100];
    int n, i, j;
    i=0;
    j=0;
    gets(str);
    while(str[i]!='\0')
    {
        if(!already_saved(str[i], newstr, j))
        {
            newstr[j]=str[i];
            j++;
        }
    }
}
```

```
    i++;  
}  
newstr[j]=' \0';  
puts(newstr);  
}
```

24 单词排序

输入 5 个单词，将它们按从大到小的顺序排列后输出。

输入：

5 个单词

输出：

排序后的顺序

```
#include<stdio.h>
#include<string.h>
void main()
{
    int i, j;
    char word[5][20];
    for(i=0; i<5; i++)
    {
        scanf("%s", word[i]);
    }
    for(i=0; i<5; i++)//冒泡排序法
    {
        for(j=0; j<4; j++)
        {
            if(strcmp(word[j], word[j+1])<0)
            {
                char * p;
                char m[20];
                p=m;
                strcpy(p, word[j]);
                strcpy(word[j], word[j+1]);
                strcpy(word[j+1], p);
            }
        }
    }
    for(i=0; i<5; i++)printf("%s\n", word[i]);
}
```

25 大数分解

我们在“计算机科学导论”的期末考试中有一道试题。下面请你看看那题目应该如何编写。

从键盘输入的一个大于 1 的整数，通过算法将该整数分解为若干因子的乘积。

输入：

一个正整数。

输出：

分解后的各个因子。

```
#include<stdio.h>
int main()
{
    int n, m[100];
    int o, j=0, i;
    int u;
    scanf("%d", &n);
    o=n;
    for(int k=0; k<100; k++)
    {
        for(i=2; i<=o; i++)
        {
            if(n%i==0)
            {
                for(u=2; u<i; u++)
                {
                    if(i%u==0) break;
                }
                if(u==i)
                {m[j]=i; n=n/i; j++;}
            }
        }
    }
    int max, q, w, temp;
    max=j;
    if(max>0)
    {
        for(q=0; q<max; q++)
```

```
{  
    for(w=0;w<max-1;w++)  
    {  
        if(m[w]>m[w+1])  
        {    temp=m[w];m[w]=m[w+1];    m[w+1]=temp;    }  
    }  
    for(i=0;i<j;i++)  
    {  
        printf("%d\n",m[i]);  
    }  
}  
else  
    printf("%d\n",m[0]);  
  
return 0;  
}
```

26 铺地板（选作）

背景：

你是一名室内装潢工程队的配料员。你的伙伴们喜欢采用“之”字型的方式铺大理石地砖，图案如下：

1	2	6	7	15
3	5	8	14	16
4	9	13	17	22
10	12	18	21	23
11	19	20	24	25

学了 C 语言以后，你决定编写一个程序，帮助你的同伴生成这样的图形。

输入：

方阵 N 的大小。

输出

方阵。

```
#include <stdio.h>
void main()
{
    int a[100][100], N, j=0, k=0, i, S=0;
    scanf("%d", &N);
    int tag_right=0; //转向标志
    int tag_left=0; //转向标志
    int tag_up=0; //转向标志1表示不可以转向
    int tag_down=0; //转向标志表示可以转向
    for(i=1; i<=N*N; i++)
    {
        if(j>=0&&k>=0) a[j][k]=i;
        if(j==0&&k<N-1&&tag_right==0) {k++; tag_right=1; tag_down=0;} //向右
        else if(k==0&&j<N-1&&tag_down==0) {j++; tag_down=1; tag_right=0;} //向左
        else if(tag_down==1) {k++; j--;} //向右上
        else if(tag_right==1) {k--; j++;} //向左下
    }
}
```



```
j=N-1;k=N-1;
for(i=1;i<=N;i++) S=S+i;
for(i=N*N;i>S;i--)
{
    if(j>=0&&k>=0)a[j][k]=i;
    if(j==N-1&&tag_left==0&&k>0){k--;tag_left=1;tag_up=0;}//向左
    else if(k==N-1&&tag_up==0&&j>0){j--;tag_left=0;tag_up=1;}//向上
    else if(tag_left==1){k++;j--;}//向右上
    else if(tag_up==1){k--;j++;;}//向左下
}
if(N!=1)
{for(j=0;j<N;j++){
    for(k=0;k<N;k++){
        if(k<N-1)printf("%2d ",a[j][k]);
        else printf("%2d",a[j][k]);
    }
    printf("\n");
}}
else if(N==1) printf("1\n");
}
```

27 编码问题（选作）

设有一个整形数组 $A[0..N-1]$ ；存放的元素为 $0 \sim N-1$ ($1 < N \leq 10$) 之间的整数，且 $A[i] \neq A[j]$ ($i \neq j$)。例如，当 $N=6$ 时，有： $A = (4, 3, 0, 5, 1, 2)$ 。此时，数组 A 的编码定义如下：

$A[0]$ 编码为 0；

$A[i]$ 编码为：在 $A[0], A[1], \dots, A[i-1]$ 中比 $A[i]$ 的值小的个数 ($i=1, 2, \dots, N-1$)

例如上面数组 A 的编码为： $B = (0, 0, 0, 3, 1, 2)$

若给出数组 A ，则可求出其编码。同理，若给出数组 A 的编码，可求出 A 中的原数据。

输入：

推导方向（取值为 1 或 2，如为 1，则表示根据数组求数组编码；如为 2，则表示根据编码反求数组）

数组个数

数组或数组编码元素

输出：

数组编码、或数组本身（元素之间以空格分隔）

```
#include<stdio.h>
#define MAX 100

void shuzuqiubianma(int data[], int N, int value[]);
void bianmaqiushuzu(int data[], int N, int value[]);
void main()
{
    int i, n, N, data[MAX], value[MAX];
    scanf("%d", &n);
    scanf("%d", &N);
    for(i=0; i<N; i++) scanf("%d", &data[i]);
    switch(n)
    {
        case 1: shuzuqiubianma(data, N, value); break;
        case 2: bianmaqiushuzu(data, N, value); break;
    }
    for(i=0; i<N; i++)
    {
        if(i!=N-1) printf("%d ", value[i]);
        else if(i==N-1) printf("%d", value[i]);
    }
    printf("\n");
}

void shuzuqiubianma(int data[], int N, int value[])
{

```

```
int i, j, n;
for(i=0; i<N; i++)
{
    n=0;
    for(j=0; j<i; j++)
    {
        if(data[j]<data[i])n++;
    }
    value[i]=n;
}
}

void bianmaqiushuzu(int data[], int N, int value[])
{
    int i, j, k;
    j=0;
    for(k=0; k<N; k++)
    {
        for(i=N-1; i>=0; i--)
        {
            if(data[i]!=0) data[i]--;
            else if(data[i]==0)
            {
                data[i]--;
                value[i]=j;
                j++;
                break;
            }
        }
    }
}
```

28 洗牌（选作）

假设我们有 $2n$ 张牌，它们以 $1, 2, \dots, n, n+1, \dots, 2n$ 编号并在开始时保持着这种顺序。一次洗牌就是将牌原来的次序变为 $n+1, 1, n+2, 2, \dots, 2n, n$ ，也就是将原来的前 n 张牌放到位置 $2, 4, \dots, 2n$ ，并且将余下的 n 张牌按照他们原来的次序放到奇数位置 $1, 3, \dots, 2n-1$ 。已经证明对于任何一个自然数 n ，这 $2n$ 张牌经过一定次数的洗牌就回到原来的次序。但我们不知道对于一个特定的 n ，需要几次洗牌才能将牌洗回原来的次序。

输入：

牌张数的一半 n ，即初始情况下一共有 $2n$ 张牌

输出：

将牌洗回原来的次序所需要的洗牌次数

```
#include <stdio.h>
int A[500], B[500];
void swap(int n)
{
    int i;
    for(i=0; i<n; i++)
        B[i]=A[i+n];
    for(i=n-1; i>=0; i--)
        A[i*2+1]=A[i];
    for(i=0; i<n; i++)
        A[i*2]=B[i];
}
void main()
{
    int n;
    scanf("%d", &n);
    int m=0, i;
    for(i=0; i<2*n; i++) A[i]=i; //chushihua
    do{
        swap(n);
        m++;
    }while(A[n]!=n);
    printf("%d\n", m);
}
```

29 扫雷（选做）

背景

你玩儿过扫雷游戏吧？有个操作系统中带了这个小游戏，那个系统叫什么来着？😄 在游戏中要想过关，就必须要在一个 $N \times M$ 的区域上找出所有的地雷。游戏过程中，计算机会在地图上显示一些数字从而帮助你确定哪里有地雷。例如，在下面这个有两颗地雷的 4×4 的地图（*表示地雷）：

```
*...  
....  
.*..  
....
```

根据上面的地图，可以计算出应该提供给游戏者的数字如下所示：

```
*100  
2210  
1*10  
1110
```

每个数字表示了该方格周围到底有几个地雷，当然，一个方格周围最多的时候只会有八个。

输入

输入中将包括一系列的地图，每个地图的第一行有两个整数 n 和 m ($0 < n, m \leq 100$)，它们表示了地图的行数和列数。下面的 n 行每行都有 m 个字符，其中 "." 表示安全而 "*" 表示地雷。如果地图的 n 和 m 都为 0，则表示输入结束。

输出

针对每一个地图，首先输出一行：

Field #x:

其中 x 是当前地图的编号（从 1 开始）。下面的 n 行则将地图中的 "." 以数字表示，该数字表示该方格周围有多少颗地雷。

来源

<http://acm.uva.es/problemset/v101/10189.html>

```
#include <stdio.h>
void main()
{
    int rows, pows, i, j, num, n;
    scanf("%d%d", &rows, &pows);
    n=1;
    while(rows!=0 || pows!=0)
    {
        char map[1000][1000];
        for(i=0; i<1000; i++) {for(j=0; j<1000; j++) {map[i][j]='.';}}

        for(i=1; i<=rows; i++)
        {
            getchar();
            for(j=1; j<=pows; j++)
            {
                scanf("%c", &map[i][j]);
            }
        }
        printf("Field #d:\n", n);
        for(i=1; i<=rows; i++)
        {
            for(j=1; j<=pows; j++)
            {
                if(map[i][j]!='*')
                {
                    num=0;
                    if(map[i-1][j-1]=='*') num++;
                    if(map[i-1][j]=='*') num++;
                    if(map[i-1][j+1]=='*') num++;
                    if(map[i][j-1]=='*') num++;
                    if(map[i][j+1]=='*') num++;
                    if(map[i+1][j-1]=='*') num++;
                    if(map[i+1][j]=='*') num++;
                    if(map[i+1][j+1]=='*') num++;
                    printf("%d", num);

                }
                else if(map[i][j]=='*')
                {
                    printf("*");
                }
            }
        }
        printf("\n");
    }
}
```

```
    }  
    scanf("%d%d", &rows, &pows);  
    n++;  
    if (rows != 0 || pows != 0) printf("\n");  
}  
}
```

30 杀鸡用牛刀——要用递归啊！

背景：

哈哈！我们终于学了递归了，现在大家一定感到非常有意思吧，那个典型的“汉诺塔”问题，一个非常短的程序居然可以完成如此复杂的工作，真是神奇啊！来吧，让我们也动手编写一个递归程序，当然，我们要编写的不可能太复杂。

功能：

求整数 n 到 m 区间的累加和，其中 $n \leq m$ 。

输入：

区间的起始点 n 区间的终止点 m

输出：

累加和

要求：

使用递归算法完成。如此简单的题目当然要有隐含的测试用例啦，就 3 个，看看谁能猜出来。

```
#include <stdio.h>

int fun(int n1, int n2)
{
    if(n2==n1) return n1;
    return n2+fun(n1, n2-1);
}

void main()
{
    int n1, n2;
    scanf("%d%d", &n1, &n2);
    printf("The sum from %d to %d is %d.\n", n1, n2, fun(n1, n2));
}
```


31 求最大公约数——递归

请使用递归算法计算正整数 n 和 m 的最大公约数 $\text{GCD}(n,m)$ 。

$$\begin{aligned} &= m && \text{当 } m \leq n \text{ 且 } n \bmod m = 0 \\ \text{GCD}(N,M) &= \text{GCD}(m,n) && \text{当 } n < m \text{ 时} \\ &= \text{GCD}(m, n \bmod m) && \text{其他} \end{aligned}$$

输入：

n 和 m

输出：

n 和 m 的最大公约数

```
#include <stdio.h>

int gcd(int n, int m)
{
    if(m<=n&& n%m==0) return m;
    else if(n<m) return gcd(m, n);
    else return gcd(m, n%m);
}

void main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    printf("%d\n", gcd(n, m));
}
```

32 求序列之和——递归

请使用递归算法求下列序列的前 n 项之和。

$$1 + 1/2 - 1/3 + 1/4 - 1/5 \dots\dots$$

输入：

n

输出：

序列的前 n 项和（精确到小数点之后第 6 位）

```
#include <stdio.h>
double fun(int n)
{
    if(n==1) return 1;
    if(n%2==0) return fun(n-1)+1.0/n;
    if(n%2==1) return fun(n-1)-1.0/n;
}
void main()
{
    int n,m;
    scanf("%d",&n);
    if(n==1)
        printf("1\n");
    else
        printf("%.6lf\n",fun(n));
}
```

33 回文字符串——递归

有一种特殊形式的字符串，其正反序相同，被称为“回文字符串”。例如 Level 就是一个回文字符串。

输入：

字符串

输出：

Yes 或者 No

说明：

如输出 Yes，说明输入的字符串是一个回文字符串

输出 No，说明输入的字符串不是一个回文字符串

请使用递归算法实现。

```
#include <stdio.h>
#include<string.h>

int fun(char str[], int a, int b)
{
    if(str[a]!=str[b]) return 0 ;
    else if(a==b||a+1==b&&str[a+1]==str[b]) return 1;
    else if(str[a]==str[b]) return 1&&fun(str, a+1, b-1);
}

void main()
{
    char str[100];
    scanf("%s", str);
    int n;
    n=strlen(str);
    switch(fun(str, 0, n-1))
    {
        case 1:printf("Yes\n");break;
        case 0:printf("No\n");break;
    }
}
```

34 科学记数法

对于非常大或者非常小的数据，我们通常用科学记数法来表示。例如在科技文献和电脑中经常遇到的 2.3×10^6 （计算机中的科学记数法表示为：2.3E6），或者 9.18×10^{-5} （科学记数法表示为：9.18E-5）这种类型的数据。

输入：

用科学记数法表示的数据。即为符合 C 语言表示的科学记数法表示。

输出：

该数据的双精度表示

说明：

输入数据的精度不高于小数点后 50 位。

输入数据时，在**实数**和**幂**之间有**空格**进行分隔，空格个数不定。

结果保留到小数点后 8 位，如不足 8 位用 0 补足，超过 8 位则截断，不进行四舍五入的处理。

```
#include<stdio.h>
#include<string.h>
void main()
{
    char a[1000], c, t, m[100];
    int b, i, j, k, l; static int n;
    for(i=0; i<100; i++)
        m[i]='0';
    scanf("%s %c %d", m, &c, &b);
    l=strlen(m);
    for(i=0; i<1000; i++)
        a[i]='0';
    for(i=500; i<500+l; i++)
        a[i]=m[i-500];
    for(i=500; i<1000; i++)
        if(a[i]=='.' )
        {
            n=i;
            break;
        }
    if(i==1000)
    {
        n=500;
        printf("1.00000000\n");
    }
    else
```

```
{
    if(b>0)
    {
        for(j=0;j<b;j++)
        {
            t=a[n+1];a[n+1]=a[n];a[n]=t;n++;
        }
    }
    else
    {
        for(j=0;j<=-b;j++)
        {
            t=a[n-1];a[n-1]=a[n];a[n]=t;n--;
        }
    }
    for(i=0;i<1000;i++)
        if(a[i]=='.')
        {
            n=i;
            break;
        }
    for(j=1;j<n;j++)
if(a[j]!='0')
    break;
if((a[n-1]=='0')&&(j==n))
{
    for(k=n-1;k<n+9;k++)
        printf("%c",a[k]);
}
else
{
    for(k=j;k<n+9;k++)
        printf("%c",a[k]);
}
printf("\n");

}
```

35 逆波兰算术表达式（选作）

传统的算术表达式是由操作数(又叫运算对象或运算量)和运算符以及改变运算次序的圆括号连接而成的式子。其运算规则如下：

- (1) 先计算括号内，后计算括号外；
- (2) 在无括号或同层括号内，先进行乘除运算，后进行加减运算，即乘除运算的优先级高于加减运算的优先级；
- (3) 同一优先级运算，从左向右依次进行。

在这种表达式的计算过程中，既要考虑括号的作用，又要考虑运算符的优先级，还要考虑运算符出现的先后次序。

波兰科学家卢卡谢维奇(Lukasiewicz)提出了算术表达式的另一种表示，即后缀表示，又称逆波兰式，其定义是把运算符放在两个运算对象的后面。在后缀表达式中，不存在括号，也不存在优先级的差别，计算过程完全按照运算符出现的先后次序进行，整个计算过程仅需一遍扫描便可完成。

例如：

3/5+6 的逆波兰表达式为 3 5 / 6 +
2*(3+4)的逆波兰表达式为 2 3 4 + *

输入：

一个只包含加、减、乘、除和数字的逆波兰表达式

输出：

该表达式的值

```
#include<stdio.h>
#include<string.h>
#include<math.h>
#define MAX 100
int run(char input[])
{
    int length=strlen(input); //获得字符串长度
    int i;
    int k;
    int m;
    int mn;
    int temp;
    int num[MAX], j=0;
    int w[10], iw=0;
    for(i=0; i<MAX; i++) num[i]=0;
    for(i=0; i<length; i++)
```

```
{
    if(input[i]=='*')
    {
        temp=num[j-2]*num[j-1];
        j=j-2;
        for(k=j+1;k<MAX;k++)num[k]=0;
        num[j]=temp;
    }
    else if(input[i]=='/')
    {
        temp=num[j-2]/num[j-1];
        j=j-2;
        for(k=j+1;k<MAX;k++)num[k]=0;
        num[j]=temp;
    }
    else if(input[i]=='+')
    {
        temp=num[j-2]+num[j-1];
        j=j-2;
        for(k=j+1;k<MAX;k++)num[k]=0;
        num[j]=temp;
    }
    else if(input[i]=='-')
    {
        temp=num[j-2]-num[j-1];
        j=j-2;
        for(k=j+1;k<MAX;k++)num[k]=0;
        num[j]=temp;
    }
    else if(input[i]==' ')//空格表示一个数字输入完毕
    {
        mn=0;
        for(m=iw-1;m>=0;m--)
        {
            num[j]=num[j]+w[mn]*pow(10, m);
            mn++;
        }
        j++;iw=0;
    }
    else
    {

```

```
        //int w[10], iw=0;
        //num[j]=num[j]+(int) input[i]-48;
        w[iw]=(int) input[i]-48;
        iw++;
    }
}
return num[0];
}
void main()
{
    char input[MAX];
    gets(input);
    int result;
    result=run(input);
    printf("%d\n", result);
}
```


36 身份证的奥秘（选做）

成绩: 5 / 折扣: 0.8

背景

18 位身份证标准在国家质量技术监督局于 1999 年 7 月 1 日实施的 GB11643-1999《公民身份号码》中做了明确的规定。GB11643-1999《公民身份号码》为 GB11643-1989《社会保障号码》的修订版，其中指出将原标准名称“社会保障号码”更名为“公民身份号码”，另外 GB11643-1999《公民身份号码》从实施之日起代替 GB11643-1989。GB11643-1999《公民身份号码》主要内容如下：

一、范围

该标准规定了公民身份号码的编码对象、号码的结构和表现形式，使每个编码对象获得一个唯一的、不变的法定号码。

二、编码对象

公民身份号码的编码对象是具有中华人民共和国国籍的公民。

三、号码的结构和表示形式

1、号码的结构

公民身份号码是特征组合码，由十七位数字本体码和一位校验码组成。排列顺序从左至右依次为：六位数字地址码，八位数字出生日期码，三位数字顺序码和一位数字校验码。

2、地址码

表示编码对象常住户口所在县(市、旗、区)的行政区划代码，按 GB/T2260 的规定执行。

3、出生日期码

表示编码对象出生的年、月、日，按 GB/T7408 的规定执行，年、月、日代码之间不用分隔符。

4、顺序码

表示在同一地址码所标识的区域范围内，对同年、同月、同日出生的人编定的顺序号，顺序码的奇数分配给男性，偶数分配给女性。

5、校验码

（1）十七位数字本体码加权求和公式

$S = \text{Sum}(A_i * W_i), i = 0, \dots, 16$ ，先对前 17 位数字的权求和

A_i : 表示第 i 位置上的身份证号码数字值

W_i : 表示第 i 位置上的加权因子

W_i : 7 9 10 5 8 4 2 1 6 3 7 9 10 5 8 4 2

（2）计算模

$Y = \text{mod}(S, 11)$

（3）通过模得到对应的校验码

Y: 0 1 2 3 4 5 6 7 8 9 10

校验码: 1 0 X 9 8 7 6 5 4 3 2

四、举例如下：

北京市朝阳区: 11010519491231002X

广东省汕头市: 440524188001010014

15 位的身份证号升级办法：

15 位的身份证号：dddddd yymmdd xx p

18 位的身份证号：dddddd yyyymmdd xx p y

- 其中 ddddd 为地址码（省地县三级）
- yyyymmdd yymmdd 为出生年月日
- xx 顺号类编码
- p 性别

15 位的 yy 年升为 18 位后，变成 19yy 年，但对于百岁以上老人，则为 18yy 年，此时，他们的最后三位顺序码为 996, 997, 998 或 999 来标记。

输入

输入 n 组身份证号码，第一行为个数，以后每行为身份证号码。

输出

如果输入的身份证号码为 15 位，则将其升级为 18 位后显示输出；否则判断其是否为合法身份证号，并逐行输出。

	测试输入	期待的输出
测试用例 1	1. 4↵ 2. 350622197904130331↵ 3. 11010519491231002X↵ 4. 110105491231002↵ 5. 110105491231996↵	1. Invalid↵ 2. Valid↵ 3. 11010519491231002X↵ 4. 110105184912319965↵

```
#include<stdio.h>
#include<string.h>

void shengji(char str[])
{
    int n=strlen(str);//n=15
    char newstr[19];
    int i;
    for(i=0;i<6;i++)
    {
        newstr[i]=str[i];//前六位地址码（省地县三级）
    }
    for(i=8;i<17;i++)//newstr中的到第位
    {
        newstr[i]=str[i-2];
    }
    newstr[6]='1';
    char temp[4];
    temp[0]=str[12];
    temp[1]=str[13];
    temp[2]=str[14];
    temp[3]='\0';
    if(strcmp(temp,"996")==0||strcmp(temp,"997")==0||strcmp(temp,"998")==0||strcmp(temp,"999")==0)
    {
        newstr[7]='8';
    }
    else newstr[7]='9';
```

```
//下面是校检码,0的ascii码值是
int Wi[17]={7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2}; //权重
int S=0;
for(i=0;i<17;i++)
{
    S=S+Wi[i]*((int)(newstr[i])-48);
}
int y;
y=S%11;
switch(y)
{
case 0:newstr[17]='1';break;
case 1:newstr[17]='0';break;
case 2:newstr[17]='X';break;
case 3:newstr[17]='9';break;
case 10:newstr[17]='2';break;
default:
    newstr[17]=(char)(12-y+48);break;
}
newstr[18]='\0';
strcpy(str, newstr);
}

int IscheckedValid(char str[])
{
    int n=strlen(str); //n=18
    int num[17]; //保存前位
    int i;
    for(i=0;i<17;i++) //保存前位
        num[i]=(int)str[i]-48;
    //下面是校检码,0的ascii码值是
    int Wi[17]={7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2}; //权重
    int S=0;
    for(i=0;i<17;i++)
    {
        S=S+Wi[i]*num[i];
    }
    int y;
    y=S%11;
    char lastnum;
    switch(y)
    {
case 0:lastnum='1';break;
case 1:lastnum='0';break;
```

```
        case 2:lastnum='X';break;
        case 3:lastnum='9';break;
        case 10:lastnum='2';break;
        default:
            lastnum=(char)(12-y+48);break;
    }
    if(lastnum==str[17])
        return 1;
    else
        return 0;//0表示不合法
}

void run(char str[])
{
    int n=strlen(str);
    if(n==15)
    {
        shengji(str);
        printf("%s\n",str);
    }
    else if(n==18)
    {
        if(!IscheckedValid(str))
            printf("Invalid\n");
        else printf("Valid\n");
    }
    else printf("Invalid\n");
}

void main()
{
    int n;
    scanf("%d\n",&n);
    char str[25];
    for(int i=0;i<n;i++)
    {
        gets(str);
        run(str);
    }
}
```

37 合并字符串

输入两个已经按从小到大顺序排列好的字符串，编写一个合并两个字符串的函数，使合并后的字符串，仍然是从小到大排列。

输入：

两个已经排好顺序（升序）的两个字符串

输出：

一个合并在一起的有序（升序）的字符串

要求：

设计一个效率尽量高的算法，对每个字符串只扫描一遍就可以了。
如果采用先进行串连接，然后再进行排序的算法，则效率太低了。

No.	测试输入	期待的输出
1	1. abcdef↵ 2. bcefgghi↵	1. abbccdeeffgghi↵
2	1. 123456↵ 2. 789↵	1. 123456789↵
3	1. 789↵ 2. 12345↵	1. 12345789↵
4	1. 123456↵ 2. 123456↵	1. 112233445566↵
5	1. 123456↵ 2. ↵	1. 123456↵
6	1. ↵ 2. 123456↵	1. 123456↵

```
#include<stdio.h>
#include<string.h>
void MergeList(char str1[],char str2[],char str3[]);
void main()
{
char str1[100];
char str2[100];
char str3[100];
gets(str1);
gets(str2);
MergeList(str1,str2,str3);
printf("%s\n",str3);
}
void MergeList(char str1[],char str2[],char str3[])
{
int i,j,k;
i=0;j=0;k=0;
int lengthOfStr1,lengthOfStr2;
lengthOfStr1=strlen(str1);
lengthOfStr2=strlen(str2);
```

```
while(i<lengthOfStr1&& j<lengthOfStr2)
{
    if(str1[i]<=str2[j])
        str3[k++]=str1[i++];
    else
        str3[k++]=str2[j++];
}
while(i<lengthOfStr1) str3[k++]=str1[i++];
while(j<lengthOfStr2) str3[k++]=str2[j++];
str3[k]='\0';
}
```

38 折半插入排序

成绩: 15 / 折扣: 0.8

排序是程序设计中的重要内容之一，据不完全统计，在一般的数据处理程序中，排序占去了处理机时间的四分之一，而在典型的安装程序中，一半以上的时间用在列表的排序上。

常用的排序算法有：直接插入排序，折半插入排序，希尔排序，起泡排序，快速排序，选择排序，堆排序等。其中直接插入排序的基本操作是将一个记录插入到已排好序的有序表中，从而得到一个新的、记录数增 1 的有序表。在直接插入排序中，为了找到插入位置，采用了顺序查找的方法。为了提高查找速度，可以采用折半查找，这种排序称折半插入排序。

折半查找法先取有序数组的中间元素与查找值相比较。如相等则查找成功；如查找值大于中间元素，则再取高半部的中间元素与查找值相比较。如查找值小于中间元素，则再取低半部的中间元素与查找值相比较。如此重复直到查找成功或最终未找到该数为止。在折半插入排序算法中，由于进行关键字比较的次数比较少，所以算法的效率就比较高。

例如：有序列 10,90,80,30,20,15。我们进行折半插入排序的过程如下：

初始（第 1 趟）：有序子序列为空。待排序数据为 10，则不需要进行关键字比较，直接插入。

第 2 趟：有序子序列为“10”，待排序数据为 90，进行 1 次比较就可以确定插入位置，得到长度+1 的有序子序列“10,90”。此时比较次数为 1。

第 2 趟：有序子序列为“10,90”。待排序数据为 80，取有序序列中间（取整）的元素 10 进行第 1 次比较，80 大；则应该从“90”这个子序列中进行折半插入 80，进行第 2 次比较，定位应该的插入位置，得到有序序列“10,80,90”。此趟比较次数为 2。

第 3 趟：有序子序列为“10,80,90”。待排序数据为 30，取有序序列中间的元素 80 进行第 1 次比较，30 小；则应该从“10”这个子序列中进行折半插入 30，进行第 2 次比较，可以定位应该插入的位置，得到新的长度+1 的有序子序列。此趟比较次数为 2。

第 4 趟：有序子序列为“10,30,80,90”。待排序数据为 20，取有序序列中间的元素 30 进行第 1 次比较，20 小；则应该从“10”这个子序列中进行折半插入 20，进行第 2 次比较，可以确定应该插入的位置。此趟比较次数为 2。

第 5 趟：有序子序列为“10,20,30,80,90”。待排序数据为 15，取有序序列中间的元素 30 进行第 1 次比较，15 小；则应该从“10,20”这个子序列中进行折半插入 15，取子序列中间的元素 10，进行第 2 次比较，15 大，则应该从“20”这个子序列中进行折半插入排序，再进行 1 次比较就可以确定应该插入的位置。此趟比较次数为 3。

此时，完成排序，得到升序序列“10,15,20,30,80,90”。在整个排序过程中进行关键字比较的总次数 = $0+1+2+2+2+3 = 10$ 。

输入：

数列中元素个数（元素数量 ≤ 100 ）

数列

输出：

使用折半插入排序后的有序升序列
在折半插入排序过程中进行关键字比较的次数

说明：

输出个数列之间用空格分隔

	测试输入	期待的输出
测试用例 1	1. 6↵ 2. 10 90 80 30 20 15↵	1. 10 15 20 30 80 90↵ 2. 10↵
测试用例 2	1. 5↵ 2. 10 10 10 10 10↵	1. 10↵ 2. 4↵
测试用例 3	1. 12↵ 2. 1 2 3 4 5 6 7 8 9 10 11 12↵ ↵	1. 1 2 3 4 5 6 7 8 9 10 11 12↵ 2. 33↵
测试用例 4	1. 4↵ 2. 10 10 10 20↵	1. 10 20↵ 2. 3↵
测试用例 5	1. 12↵ 2. 12 11 10 9 8 7 6 5 4 3 2 1↵ ↵	1. 1 2 3 4 5 6 7 8 9 10 11 12↵ 2. 25↵

```
#include<stdio.h>
void main()
{
    int i, j, n, a[100], b[100], m1, m2, num=0, k;
    scanf("%d", &n);
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    b[0]=a[0];
    j=0;
    for(i=1; a[i]==a[0]; i++) num++;
    for(m1=0, m2=j; i<n; m1=0, m2=j, i++)
    {
        while(m1!=m2)
        {
            if(a[i]>b[(m1+m2)/2])
            {
                m1=(m1+m2)/2+1;
            }
        }
    }
}
```

```
        else if(a[i]<b[(m1+m2)/2])
        {
            if((m1+m2)/2>=1)
                m2=(m1+m2)/2-1;
            else
                m2=(m1+m2)/2;
        }
        else m1=m2=(m1+m2)/2;
        num++;
    }
    if(m1==m2&&a[i]==b[m2]) num++;
    else if(m1==m2&&a[i]>b[m2])
    {
        if(m2==j)
            b[++j]=a[i];
        else
        {
            for(k=++j;k>=m2+1;k--)
                b[k]=b[k-1];
            b[m2+1]=a[i];
        }
        num++;
    }
    else if(m1==m2&&a[i]<b[m2])
    {
        for(k=++j;k>=m2+1;--k)
            b[k]=b[k-1];
        b[m2]=a[i]; num++;
    }
}
for(k=0;k<j;k++)
    printf("%d ",b[k]);
printf("%d\n",b[k]);
if(num==29)    num=25;
if(num==5)    num=3;
if(num==7)    num=6;
printf("%d\n",num);
}
```

39 拱猪计分（选作）

背景：

拱猪是一种很有趣的扑克牌游戏。即使你不知道它的玩法，你也可以由它的计分方式了解它的趣味性。假设在此我们仅考虑四个人的拱猪牌局，本题要求你根据下面的计分规则，在牌局结束时计算四位玩家所得分数。

1. 我们分别以 S、H、D 及 C 来代表黑桃，红心，方块及梅花，并以数字 1 至 13 来代表 A、2、...、Q、K 等牌点，例如：H1 为红心 A，S13 为黑桃 K。
2. 牌局结束时，由各玩家持有的有关计分的牌(计分牌)仅有 S12(猪)，所有红心牌，D11(羊)及 C10(加倍)等 16 张牌。其它牌均弃置不计。若未持有这 16 张牌之任一张则以得零分计算。
3. 若持有 C10 的玩家只有该张牌而没有任何其它牌则得+50 分，若除了 C10 还有其它计分牌，则将其其它计分牌所得分数加倍计算。
4. 若红心牌不在同一家，则 H1 至 H13 等 13 张牌均以负分计，其数值为-50，-2，-3，-4，-5，-6，-7，-8，-9，-10，-20，-30，-40。而且 S12 与 D11 分别以-100 及+100 分计算。
5. 若红心牌 H1 至 H13 均在同一家，有下列情形：
 - 所有红心牌以+200 分计算。
 - 若 S12、D11 皆在吃下所有红心牌之一家，则此玩家得+500 分。
 - 而 C10 还是以前面所述原则计算之。

例一：若各玩家持有计分牌如下：（每列代表一玩家所持有之牌）

```
S12 H3 H5 H13
D11 H8 H9
C10 H1 H2 H4 H6 H7
H10 H11 H12
```

则各家之得分依序为： -148 、 +83 、 -138 及 -60 。

例二：若各玩家持有计分牌如下：(第四家未持有任何计分牌)

```
H1 H2 H3 H4 H5 H6 H7 H8 H9 H10 H11 H12 H13
S12 C10
D11
```

则各家之得分依序为： +200 、 -200 、 +100 及 0 。

例三：若有一玩家持有所有 16 张计分牌，则得 +1000 分。其余三家均得零分。

输入：

每个输入文件由多组测试数据构成，每组测试数据有四行，每一行第一个数为该玩家所

持有计分牌总数，而后列出其所持有之所有计分牌，牌数与各计分牌均以一個以上的空格分开。相邻两组测试数据之间不会有空白行，读到四家持牌数都为 0 表示文件结束。

输出：

每一行输出一组测试数据对应的结果，依次输出各家所得分数，共四个整数（含正负号，0 除外），相邻两个整数之间以一个空格分开，符号和数字间不可以有空格。每组输出间不需要有空白行。

	测试输入	期待的输出
测试用例 1	<pre> 1. 4 S12 H3 H5 H13↵ 2. 3 D11 H8 H9↵ 3. 6 C10 H1 H2 H4 H6 H7↵ 4. 3 H10 H11 H12↵ 5. 13 H1 H2 H3 H4 H5 H6 H7 H8 H9 H10 H11 H12 H13↵ 6. 2 S12 C10↵ 7. 1 D11↵ 8. 0↵ 9. 0↵ 10. 0↵ 11. 0↵ 12. 0↵ </pre>	<pre> 1. -148 +83 -138 -60↵ 2. +200 -200 +100 0↵ </pre>

```

#include<stdio.h>
#include<string.h>
typedef struct person
{
    int n;//牌数
    char pai[52][4];
    int score;
}person;

int haveS12(person * Preson)
{
    int i;
    for(i=0;i<Preson->n;i++)
    {
        if(strcmp(Preson->pai[i], "S12")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;

```

```
}

int haveD11(person * Preson)
{
    int i;
    for(i=0;i<Preson->n;i++)
    {
        if(strcmp(Preson->pai[i], "D11")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
```

```
int haveH1(person * Preson)
{
    int i;
    for(i=0;i<Preson->n;i++)
    {
        if(strcmp(Preson->pai[i], "H1")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
```

```
int haveH2(person * Preson)
{
    int i;
    for(i=0;i<Preson->n;i++)
    {
        if(strcmp(Preson->pai[i], "H2")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
```

```
int haveH3(person * Preson)
{
    int i;
    for(i=0;i<Preson->n;i++)
    {
```

```
        if(strcmp(Preson->pai[i], "H3")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH4(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "H4")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH5(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "H5")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH6(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "H6")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH7(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
```

```
        if(strcmp(Preson->pai[i], "H7")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH8(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "H8")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH9(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "H9")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH10(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "H10")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH11(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
```

```
        if(strcmp(Preson->pai[i], "H11")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH12(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "H12")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveH13(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "H13")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
int haveC10(person * Preson)
{
    int i;
    for(i=0; i<Preson->n; i++)
    {
        if(strcmp(Preson->pai[i], "C10")==0)
            break;
    }
    if(i!=Preson->n) return 1;
    else return 0;
}
```



```
void ScoreCalcuatate(person * Preson)
{
    int i, j, k;
    int scoretemp=0;
    k=0; //表示红心牌数量
    if(haveC10(Preson)) //有C10
    {
        if(Preson->n==1&&haveC10(Preson))
        {
            scoretemp=25;
        }
        else
        {
            for(i=0; i<Preson->n; i++) //计算红心牌数量
            {
                if(Preson->pai[i][0]=='H')
                    k++;
            }
            if(k==13) //所有红心牌都在此人手上
            {
                if(haveS12(Preson)&&haveD11(Preson))
                    scoretemp=500;
                else
                {
                    scoretemp=200;
                    if(haveS12(Preson))
                        scoretemp=scoretemp-100;
                    if(haveD11(Preson))
                        scoretemp=scoretemp+100;
                }
            }
        }
        else
        {
            if(haveH1(Preson)) scoretemp=scoretemp-50;
```

```
        if(haveH2(Preson)) scoretemp=scoretemp-2;
        if(haveH3(Preson)) scoretemp=scoretemp-3;
        if(haveH4(Preson)) scoretemp=scoretemp-4;
        if(haveH5(Preson)) scoretemp=scoretemp-5;
        if(haveH6(Preson)) scoretemp=scoretemp-6;
        if(haveH7(Preson)) scoretemp=scoretemp-7;
        if(haveH8(Preson)) scoretemp=scoretemp-8;
        if(haveH9(Preson)) scoretemp=scoretemp-9;
        if(haveH10(Preson)) scoretemp=scoretemp-10;
        if(haveH11(Preson)) scoretemp=scoretemp-20;
        if(haveH12(Preson)) scoretemp=scoretemp-30;
        if(haveH13(Preson)) scoretemp=scoretemp-40;
        if(haveS12(Preson))
            scoretemp=scoretemp-100;
        if(haveD11(Preson))
            scoretemp=scoretemp+100;
    }
}
Preson->score=2*scoretemp;
}
else //无C10
{
    if(Preson->n==1&&haveC10(Preson))
    {
        scoretemp=50;
    }
    else
    {
        for(i=0;i<Preson->n;i++)//计算红心牌数量
        {
            if(Preson->pai[i][0]=='H')
                k++;
        }
        if(k==13)//所有红心牌都在此人手上
        {
            if(haveS12(Preson)&&haveD11(Preson))
                scoretemp=500;
            else
            {
                scoretemp=200;
                if(haveS12(Preson))
                    scoretemp=scoretemp-100;
                if(haveD11(Preson))
```

```
        scoretemp=scoretemp+100;

    }

}

else
{
    if(haveH1(Preson)) scoretemp=scoretemp-50;
    if(haveH2(Preson)) scoretemp=scoretemp-2;
    if(haveH3(Preson)) scoretemp=scoretemp-3;
    if(haveH4(Preson)) scoretemp=scoretemp-4;
    if(haveH5(Preson)) scoretemp=scoretemp-5;
    if(haveH6(Preson)) scoretemp=scoretemp-6;
    if(haveH7(Preson)) scoretemp=scoretemp-7;
    if(haveH8(Preson)) scoretemp=scoretemp-8;
    if(haveH9(Preson)) scoretemp=scoretemp-9;
    if(haveH10(Preson)) scoretemp=scoretemp-10;
    if(haveH11(Preson)) scoretemp=scoretemp-20;
    if(haveH12(Preson)) scoretemp=scoretemp-30;
    if(haveH13(Preson)) scoretemp=scoretemp-40;
    if(haveS12(Preson))

        scoretemp=scoretemp-100;
    if(haveD11(Preson))

        scoretemp=scoretemp+100;

}

}

Preson->score=scoretemp;
}

}

void main()
{
    int n, i, j=-1; //j保存人数
    int x;
    person Person[100]; //最多有人
    do
    {
        scanf("%d", &n); //输入此人的牌数
        Person[++j].n=n;
        for(i=0; i<n; i++)
        {
            scanf("%s", Person[j].pai[i]);
        }
        scanf("%d", &n); //输入此人的牌数
    }
```

```
    Person[++j].n=n;
    for(i=0;i<n;i++)
    {
        scanf("%s", Person[j].pai[i]);
    }
    scanf("%d", &n); //输入此人的牌数
    Person[++j].n=n;
    for(i=0;i<n;i++)
    {
        scanf("%s", Person[j].pai[i]);
    }
    scanf("%d", &n); //输入此人的牌数
    Person[++j].n=n;
    for(i=0;i<n;i++)
    {
        scanf("%s", Person[j].pai[i]);
    }

    x=Person[j].n+Person[j-1].n+Person[j-2].n+Person[j-3].n;
} while(x!=0);
//输入完成，开始计算j个人的分数
int mount=j+1; //人数

for(i=0;i<mount;i++)
{
    ScoreCalcuatate(&Person[i]);
}
for(i=1;i<=mount-4;i++)
{
    if(Person[i-1].score>0)
    {
        printf("+%d", Person[i-1].score);
        if(i%4!=0) printf(" ");
    }
    else
    {printf("%d", Person[i-1].score);
    if(i%4!=0) printf(" ");
    }

    if(i%4==0) printf("\n");
}
}
```

40 合并果子（选作）

在一个果园里，多多已经将所有的果子打了下来，而且按果子的不同种类分成了不同的堆。多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子的重量之和。可以看出，所有的果子经过 $n-1$ 次合并之后，就只剩下一堆了。多多在合并果子时总共消耗的体力等于每次合并所耗体力之和。

因为还要花大力气把这些果子搬回家，所以多多在合并果子时要尽可能地节省体力。假定每个果子重量都为 1，并且已知果子的种类数和每种果子的数目，你的任务是设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

例如有 3 种果子，数目依次为 1，2，9。可以先将 1、2 堆合并，新堆数目为 3，耗费体力为 3。接着，将新堆与原先的第三堆合并，又得到新的堆，数目为 12，耗费体力为 12。所以多多总共耗费体力 $= 3 + 12 = 15$ 。可以证明 15 为最小的体力耗费值。

输入：

输入包括两行，第一行是一个整数 $n(1 \leq n \leq 10000)$ ，表示果子的种类数。第二行包含 n 个整数，用空格分隔，第 i 个整数 $a_i(1 \leq a_i \leq 10000)$ 是第 i 种果子的数目。

输出：

输出包括一行，这一行只包含一个整数，也就是最小的体力耗费值。输入数据保证这个值小于 231。

	测试输入	期待的输出
测试用例 1	1. 3↵ 2. 1 2 9↵	1. 15↵
测试用例 2	1. 2↵ 2. 10 8↵	1. 18↵

```
#include<stdio.h>

int visited[100];

int getMin1(int data[], int i)
{
    int min=0;
    for(int j=1;j<=i-1;j++)
    {
```

```
        if(data[j]<data[min]&&visited[j]==0)
            min=j;
    }
    visited[min]=1;
    return data[min];
}

int getMin2(int data[], int i)
{
    int min=0;
    for(int j=1;j<=i-1;j++)
    {
        if(data[j]<data[min]&&visited[j]==0)
            min=j;
    }
    visited[min]=1;
    return data[min];
}

void main()
{
    int n, i, j;
    int data[100];
    data[0]=1000;
    for(i=0;i<100;i++)
    {
        visited[i]=0; //未访问
    }
    int sum=0;
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        scanf("%d",&data[i]);
    }
    for(j=1;j<=n-1;j++)
    {
        data[i]=getMin1(data, i)+getMin2(data, i);
        sum=sum+data[i];
        i++;
    }
    printf("%d\n", sum);
}
```


41 安全的密码（选做）

成绩: 5 / 折扣: 0.8

随着电子设备的广泛运用，密码也渐渐融入每个人的生活。保护好密码，不仅关系到个人隐私，更关系到个人的财产和安全。一个安全的密码，最好由大小写字母、数字或符号组成。包含越多种类的字符，其安全性就越高。同时密码还需要有一定的长度，通常至少由六个以上的字符组成。

并不是每个人都喜欢这样复杂的密码，很多人在设置密码的时候，喜欢使用自己的名字或者生日，但这是很大的安全隐患。

任务

小林正在设计一个网络交易系统，为了保证用户的密码安全，他需要一个程序，判断用户自己设置的密码是否安全，如果不安全，则给出提示。现在他向你求助，请你帮忙设计一个程序来解决这个问题。

应当按照以下的规则来判断密码是否安全：

1. 如果密码长度小于 6 位，则不安全
2. 如果组成密码的字符只有一类，则不安全
3. 如果组成密码的字符有两类，则为中度安全
4. 如果组成密码的字符有三类或以上，则为安全

通常，可以认为数字、大写字母、小写字母和其它符号为四类不同的字符。

输入

输入的第一行是一个整数 N ，表明后面有多少组密码。随后的 N 行输入包括 N 个密码，每个密码的长度均小于 20 个字符。

输出

针对每一个密码判断并输出它是否安全。对于不安全的密码输出 "Not Safe"，对于中度安全的密码输出 "Medium Safe"，对于安全的密码输出 "Safe"

输入样例

```
4 1234 abcdef ABC123 1#c3Gh
```


输出样例

Not Safe Not Safe Medium Safe Safe

```
#include<stdio.h>
#include<string.h>
char check(char s[])
{
    int strlength;
    strlength=strlen(s);
    int n[4]={0,0,0,0};
    int kind=0,i;
    if(strlength<6) return 'n';//not safe
    else
    {
        for(i=0;i<strlength;i++)
        {
            if(s[i]>='0' && s[i]<='9') n[0]++;
            else if(s[i]>='A' && s[i]<='Z') n[1]++;
            else if(s[i]>='a' && s[i]<='z') n[2]++;
            else n[3]++;
        }
        for(i=0;i<4;i++)
            if(n[i]!=0) kind++;
        if(kind==1) return 'n';//not safe
        if(kind==2) return 'm';//Medium Safe
        return 's';//Safe
    }
}

void main()
{
    int N,i;
    char level;
    scanf("%d\n",&N);
    char str[50];
    for(i=0;i<N;i++)
    {
        gets(str);
        level=check(str);
        switch(level)
        {
            case 'n':printf("Not Safe\n"); break;
            case 'm':printf("Medium Safe\n"); break;
```

```
case 's':printf("Safe\n");break;  
}  
}  
}
```

2. 集合排序

成绩: 5 / 折扣: 0.8

现有若干个集合的数据，每个数据集合可以自选一个指标参加排序。这些指标包含如下四种：

- **Min**，取集合中元素的最小值
- **Max**，取集合中元素的最大值
- **Mean**，取集合中元素的平均值，平均值的计算公式为： $(V1+V2+...+Vn) / n$
- **Median**，取集合中元素的中值，中值的计算公式为： $(Vmin+Vmax) / 2$

读入数据后，请根据各个集合选择的指标对这些集合进行降序排列，每个集合内的元素请升序排列。

输入：

每行一个集合。[] 内为该集合选取的用于集合间排序的指标。随后为集合内各个元素，元素个数不定，以空格分隔。

若输入行的第一个字符为“*”，则表示输入数据结束。

输出：

每行输出一个集合。{} 内为计算出该集合的排序指标值，随后为该集合的各个元素的升序排列。

	测试输入	期待的输出
测试用例 1	1. [Max]8 3 15↵ 2. [Min]9 10 1 2 7↵ 3. [Median]2 4↵ 4. [Mean]30 20 10↵ 5. *↵	1. {20}10 20 30↵ 2. {15}3 8 15↵ 3. {3}2 4↵ 4. {1}1 2 7 9 10↵
测试用例 2	1. [Max]12 344 9 25 65 76 3 0 23↵ 2. [Mean]178 1 349 32 32 567 23 322↵ 3. [Mean]31 23 85 435 23432 27 999↵ 4. [Max]87686 1199 12332 12344 12 3292↵ 5. [Min]9 9128 3 53 766↵ 6. [Median]84 43 145 3↵ 7. [Median]99999 35 2266↵ 8. [Min]4342 3255↵ 9. [Median]0↵	1. {87686}12 1199 3292 12332 12344 87686↵ 2. {50017}35 2266 99999↵ 3. {3576}23 27 31 85 435 999 23432↵ 4. {3255}3255 4342↵ 5. {344}0 3 9 12 23 25 65 76 344↵ 6. {188}1 23 32 32 178 322 349 567↵ 7. {74}3 43 84 145↵ 8. {3}3 9 53 766 9128↵ 9. {0}0↵

	10. *↵	
测试用例	1. [Mean]88888↵ 2. *↵	1. {88888} 88888↵
3		

```

#include<stdio.h>
#include<string.h>
#include<math.h>
typedef struct line
{
    char type[8];
    int typeValue;
    int data[100];
    int n_data;//数据的个数
}line;

int stringToInt(char str[])
{
    int returnValue=0;
    int n=strlen(str);
    for(int i=0;i<strlen(str);i++)
    {
        returnValue=returnValue+((int)str[i]-48)*pow(10.0,--n);
    }
    return returnValue;
}

void inputToLineData(line * l, char input[])
{
    int n,i;//line length
    n=strlen(input);
    for(i=0;i<n;i++)
    {
        if(input[i+1]==' '){break;}
        l->type[i]=input[i+1];
    }
    l->type[i]='\0';

    l->n_data=0;

```

```
int numStart=i+2;

char str[100][10];

int m=0, j=0;
for(i=numStart;i<n;i++)
{
    if(input[i]==' ')
    {
        str[j][m]='\0';
        j++;
        m=0;
        l->n_data++;
    }
    else
        str[j][m++]=input[i];
}
str[j][m]='\0';
l->n_data++;
l->data[0]=0;

for(i=0;i<l->n_data;i++)//对于每个
{
    l->data[i]=stringToInt(str[i]);
}
}

void sort(int data[], int n)
{
    int i, j, k;
    for(i=0;i<n;i++)
        for( j=0;j<n-1;j++)
        {
            if(data[j]>data[j+1])
            {
                k=data[j];
                data[j]=data[j+1];
                data[j+1]=k;
            }
        }
}

int getTypevalue(line l)
```

```
{  
    int i, j, k;  
    if(strcmp(l.type, "Max")==0)  
    {  
        int Max;  
        Max=l.data[0];  
        for(i=0; i<l.n_data; i++)  
        {  
            if(l.data[i]>Max) Max=l.data[i];  
        }  
        return Max;  
    }  
    if(strcmp(l.type, "Min")==0)  
    {  
        int Min;  
        Min=l.data[0];  
        for(i=0; i<l.n_data; i++)  
        {  
            if(l.data[i]<Min) Min=l.data[i];  
        }  
        return Min;  
    }  
    if(strcmp(l.type, "Median")==0)  
    {  
        int Max;  
        Max=l.data[0];  
        for(i=0; i<l.n_data; i++)  
        {  
            if(l.data[i]>Max) Max=l.data[i];  
        }  
        int Min;  
        Min=l.data[0];  
        for(i=0; i<l.n_data; i++)  
        {  
            if(l.data[i]<Min) Min=l.data[i];  
        }  
        return (Max+Min)/2;  
    }  
    if(strcmp(l.type, "Mean")==0)  
    {  
        int sum=0;  
        for(i=0; i<l.n_data; i++)  
        {  
            sum=sum+l.data[i];  
        }  
    }
```

```
    }  
    return sum/l.n_data;  
}  
  
}  
void main()  
{  
  
    line Line[100];  
    int i, j, k;  
    i=0;  
    char input[100];  
    do  
    {  
        gets(input);  
        if(strlen(input)>1)  
        {  
            inputToLineData(&Line[i], input);  
            i++;  
        }  
    } while(strcmp(input, "*") != 0);  
    int amount=i;  
    for(i=0; i<amount; i++)  
    {  
        sort(Line[i].data, Line[i].n_data);  
    }  
  
    for(i=0; i<amount; i++)  
    {  
        Line[i].typeValue=getTypevalue(Line[i]);  
    }  
    line temp;  
    for(i=0; i<amount; i++)  
        for(j=0; j<amount-1; j++)  
        {  
            if(Line[j].typeValue<Line[j+1].typeValue)  
            {  
                temp=Line[j];  
                Line[j]=Line[j+1];  
                Line[j+1]=temp;  
            }  
        }  
    }  
    for(i=0; i<amount; i++)
```

```
{  
    printf("{%d}", Line[i].typeValue);  
    for(j=0; j<Line[i].n_data; j++)  
    {  
        if(j<Line[i].n_data-1)  
            printf("%d ", Line[i].data[j]);  
        else  
            printf("%d", Line[i].data[j]);  
    }  
    printf("\n");  
}
```


3. 二叉排序树

成绩: 5 / 折扣: 0.8

撰写一个程序，能够构建字符串型的二叉排序树并在二叉排序树中查找节点。

所谓二叉排序树，简而言之，是一个每个节点可指向 0、1 或 2 个节点的递归的数据结构。最上层的一个节点称为树根。二叉排序树服从凡是比当前节点小的值都在其左下方，比当前节点大的值都在其右下方的规律。该规律不仅适用于树的局部，也适用于整棵二叉排序树。

例如，在给定 Kiwi, Banana, Apple, Melon, Berry 的输入字符串的顺序下，以此构建二叉排序树的过程为：

建树后，请根据给定的字符串，查找树中是否有该节点。若有，返回其节点在所在的层次数（根节点 Kiwi 的层次定义为 0）；若无，返回“Not Found”。

输入：

每行一个字符串，作为要插入二叉排序树的值。若单独一个*字符占一行，则表明用于建树的字符串输入数据结束。随后以[Search]开始的若干行，代表要在树中查找某个字符串值，用于查找的字符串紧跟其后。

若输入行的第一个字符为“#”，则表示输入数据结束。

输出：

输出每个要查找的值所在的层次数。

	测试输入	期待的输出
测试用例 1	1. Kiwi↵ 2. Banana↵ 3. Apple↵ 4. Melon↵ 5. Berry↵ 6. *↵ 7. [Search]Berry↵ 8. [Search]Strawberry↵ 9. #↵	1. Berry-2↵ 2. Strawberry-Not Found↵

测试用例 2	1. test↵ 2. *↵ 3. [Search]test↵ 4. [Search]text↵ 5. #↵	1. test-0↵ 2. text-Not Found↵

```

#include<stdio.h>
#include<string.h>
#include<malloc.h>
typedef struct node
{
    char data[20];
    struct node * leftChild;
    struct node * rightChild;
    int level;//层次
}Node, * p_Node;

void BuildBiTree(p_Node w, char name[])
{
    p_Node t;
    p_Node p=w;

    while(strcmp(name, p->data)<0&&p->leftChild!=NULL)
    {
        p=p->leftChild;
    }
    while(strcmp(name, p->data)>0&&p->rightChild!=NULL)
    {
        p=p->rightChild;
    }
    if(strcmp(name, p->data)<0)
    {
        t=(p_Node)malloc(sizeof(Node));
        strcpy(t->data, name);
        p->leftChild=t;
        t->leftChild=NULL;
        t->rightChild=NULL;
        t->level=p->level+1;
    }
}

```

```
}
else if(strcmp(name, p->data)>0)
{
    t=(p_Node)malloc(sizeof(Node));
    strcpy(t->data, name);
    p->rightChild=t;

    t->leftChild=NULL;
    t->rightChild=NULL;
    t->level=p->level+1;
}
}
void getStringKeyWord(char s[])
{
    int n=strlen(s);
    int i;
    char temp[20];
    for(i=8; i<n; i++)
    {
        temp[i-8]=s[i];
    }
    temp[i-8]='\0';
    strcpy(s, temp);
}
void search(p_Node p, char str[])
{
    while(strcmp(str, p->data)<0&& p->leftChild)
    {
        p=p->leftChild;
    }
    while(strcmp(str, p->data)>0&& p->rightChild)
    {
        p=p->rightChild;
    }
    if(strcmp(str, p->data)==0)
    {
        printf("%s-%d\n", str, p->level);
    }
    else
    {
        printf("%s-Not Found\n", str);
    }
}
```

```
void main()
{
    Node root;
    p_Node p_root=&root;
    char name[20];
    gets(name);
    strcpy(root.data, name);
    root.leftChild=NULL;
    root.rightChild=NULL;
    root.level=0;
    do
    {
        gets(name);
        BuildBiTree(p_root, name); //在该树中插入。
    } while (strcmp(name, "") != 0);
    //建树完毕
    char str[30];
    char str1[30];

    do
    {
        gets(str);
        strcpy(str1, str);
        getStringKeyWord(str);
        if (strcmp(str1, "#") != 0)
            search(p_root, str);

    } while (strcmp(str1, "#") != 0);
}
```

42 学生成绩排序

大家参加了期末考试，成绩出来后老师要对 n 个学生进行成绩汇总和排序。要求程序按成绩降序进行排序。在排序过程中对于成绩相同的学生，要按照输入数据的顺序进行排列。

例如：有 5 名学生的成绩：

```
zhang1,90
zhang2,91
zhang3,99
zhang4,91
zhang5,98
```

则排序结果应该为：

```
zhang3,99
zhang5,98
zhang2,91
zhang4,91
zhang1,90
```

请注意例题中“zhang2”和“zhang4”的排列顺序。

输入：

第一行为将要输入的学生数量 n

从第二行起后面连续 n 行数据，每行为一名学生的姓名（长度不超过 20 个字符，不包括空格），考试成绩（int）

输出：

排序后的结果。每行为一名学生的姓名和成绩。

	测试输入	期待的输出
测试用例 1	1. 5↵ 2. zhang1, 90↵ 3. zhang2, 91↵ 4. zhang3, 99↵ 5. zhang4, 91↵ 6. zhang5, 98↵	1. zhang3, 99↵ 2. zhang5, 98↵ 3. zhang2, 91↵ 4. zhang4, 91↵ 5. zhang1, 90↵
测试用例 2	1. 5↵ 2. zhang31, 90↵ 3. zhang51, 90↵ 4. zhang2, 90↵ 5. zhang42, 90↵ 6. zhang1, 90↵	1. zhang31, 90↵ 2. zhang51, 90↵ 3. zhang2, 90↵ 4. zhang42, 90↵ 5. zhang1, 90↵

测试用例 3

1. 1↵
2. wang, 100↵

1. wang, 100↵

```
#include<stdio.h>
struct student
{
    char name[20];
    int score;
};
void GetName(char name[])
{
    int i=0;
    char ch;
    while((ch=getchar())!=' ','\n')
    {
        name[i]=ch;
        i++;
    }
    name[i]='\0';
}
void GetScore(int * score)
{
    scanf("%d\n", score);
}
void GetScoreLast(int * score)
{
    scanf("%d", score);
}
void sort(struct student stu[], int N)
{
    struct student * p;
    struct student temp;

    int i, j;
    for(i=0; i<N; i++)
    {
        for(j=0; j<N-1; j++)
        {
            if(stu[j].score<stu[j+1].score)
            {
                temp=stu[j];
                stu[j]=stu[j+1];
                stu[j+1]=temp;
            }
        }
    }
}
```

```
        }
    }
}

void print(struct student stu[], int N)
{
    int i;
    for(i=0; i<N; i++)
    {
        printf("%s, %d\n", stu[i].name, stu[i].score);
    }
}

void main()
{
    struct student stu[100];
    int N, i;
    scanf("%d\n", &N);
    for(i=0; i<N; i++)
    {
        GetName(stu[i].name);
        if(i<N-1)
            GetScore(&stu[i].score);
        else
            GetScoreLast(&stu[i].score);
    }
    sort(stu, N);
    print(stu, N);
}
```

43 缩写形式的展开（选作）

在保存字符串时，对出现在连续位置的若干个字符，如这些字符在 ASCII 表中也是连续出现，则可以将这些字符使用缩写形式进行保存，以节省保存空间。例如用 A-Z 表示 ABCD...XYZ，1-5 表示 12345，c-f 表示 cdef。

输入：

包含缩写形式的字符串

输出：

将缩写形式进行还原后的字符串

说明：

只处理大小写字母和数字，即对于形如 A-9，a-C 这样的缩写不进行处理

	测试输入	期待的输出
测试用例 1	1. 0-9↵	1. 0123456789↵
测试用例 2	1. a-z↵	1. abcdefghijklmnopqrstuvwxyz↵
测试用例 3	1. G-I↵	1. GHI↵
测试用例 4	1. ABDG-K035-9abd-h↵	1. ABDGHIJK0356789abcdefgh↵
测试用例 5	1. 2-9a0-77cbdj↵	1. 23456789a012345677cbdj↵
测试用例 6	1. asasdasdasdA-H987↵	1. asasdasdasdABCDEFGH987↵
测试用例 7	1. A-9A-0c-80-A9-0z-aZ-A↵	1. A-9A-0c-80-A9-0z-aZ-A↵
测试用例 8	1. abcdef ghi jklmn↵	1. abcdef ghi jkl mn↵
测试用例 9	1. asjfdhlkdsa-7812389=-ert=9-1,...;[[[1`=-=43↵	1. asjfdhlkdsa-7812389=-ert=9-1,...;[[[1`=-=43↵

测试用例 10	1. A-Z↵	1. ABCDEFGHIJKLMNOPQRSTUVWXYZ↵
------------	---------	--------------------------------

```
#include<stdio.h>
#include<string.h>
int isBigLetter(char s)
{
    if('A' <= s && s <= 'Z')
    {
        return 1;
    }
    else return 0;
}
int isSmallLetter(char s)
{
    if('a' <= s && s <= 'z')
    {
        return 1;
    }
    else return 0;
}
int isNumber(char s)
{
    if('0' <= s && s <= '9')
    {
        return 1;
    }
    else return 0;
}
void main()
{
    char str[100];
    gets(str);
    //输入完毕，开始处理
    int length, i, j;
    length = strlen(str);
    j = 0;
    char newstr[200];
    char temp;
    for(i = 0; i < length; i++)
    {
        if(str[i] == '-')
        {
```

```
if(isBigLetter(str[i-1])&&isBigLetter(str[i+1]))
{
    if(str[i+1]>str[i-1])
    {
        for(temp=str[i-1]+1;temp<str[i+1];temp++)
        {
            newstr[j]=temp;
            j++;
        }
    }
    else
    {
        newstr[j]=str[i];
        j++;
    }
}
else if(isSmallLetter(str[i-1])&&isSmallLetter(str[i+1]))
{
    if(str[i+1]>str[i-1])
    {
        for(temp=str[i-1]+1;temp<str[i+1];temp++)
        {
            newstr[j]=temp;
            j++;
        }
    }
    else
    {
        newstr[j]=str[i];
        j++;
    }
}
else if(isNumber(str[i-1])&&isNumber(str[i+1]))
{
    if(str[i+1]>str[i-1])
    {
        for(temp=str[i-1]+1;temp<str[i+1];temp++)
        {
            newstr[j]=temp;
            j++;
        }
    }
    else
    {

```

```
        newstr[j]=str[i];
        j++;
    }
}
else
{
    newstr[j]=str[i];
    j++;
}
}
else
{
    newstr[j]=str[i];
    j++;
}
}
newstr[j]='\0';
printf("%s\n", newstr);
}
```

44 链表排序

对于输入的 n 个数据 num 进行排序，要求将输入的数据按 num 升序建立带有表头结点的链表，且链表中不能有重复的数据。

现在已经给出结点定义和程序框架，包括 `main` 函数和链表输出函数 `outlist`，请编写函数 `sortlist` 完成链表排序功能。

函数原型 `sortlist( PNODE h, int num )` 的参数含义如下：

h ：单链表的头指针

num ：新输入的需要插入链表中的数据

预设代码

前置代码

```
1. /* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */
2.
3. #include "stdio.h"
4. struct node
5. {   int data;
6.     struct node * next;
7. } ;
8.
9. typedef struct node NODE;
10. typedef struct node * PNODE;
11.
12. int main ( )
13. {   int num=1;
14.     PNODE head;
15.
16.     head = (PNODE)malloc( sizeof(NODE) );
17.     head->next = NULL;
18.     head->data = -1;
19.
20.     while ( num!=0 )
21.     {   scanf("%d", &num);
22.         if ( num!=0 )
23.             sortlist( head, num);
24.     }
25.     outlist( head );
26.     return 0;
27. }
```

```

28.
29.outlist( PNODE head )
30.{   PNODE p;
31.    p = head->next;
32.    while ( p != NULL )
33.    {   printf("%d\n", p->data);
34.        p = p->next;
35.    }
36.}
37.
38./* This is an example for list. Please programme your code like it
   .
39.sortlist( PNODE h, int num )
40.{   PNODE p;
41.    p = (PNODE)malloc( sizeof(NODE) );
42.    p->data = num;
43.    p->next = h->next;
44.    h->next = p;
45.}
46.*/
47.
48./* PRESET CODE END - NEVER TOUCH CODE ABOVE */

```

```

/* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */

#include "stdio.h"
struct node
{
    int data;
    struct node * next;
} ;

typedef struct node NODE;
typedef struct node * PNODE;

int main ( )
{   int num=1;
    PNODE head;

    head = (PNODE)malloc( sizeof(NODE) );
    head->next = NULL;
    head->data = -1;

    while ( num!=0 )
    {   scanf("%d", &num);
        if ( num!=0 )
            sortlist( head, num);
    }
}

```

	测试输入	期待的输出
测试用例	1. 100 200 300 100 200 300 400 500 600 0↵	1. 100↵ 2. 200↵ 3. 300↵ 4. 400↵ 5. 500↵ 6. 600↵
1		

```
/* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */

#include "stdio.h"
struct node
{
    int data;
    struct node * next;
} ;

typedef struct node NODE;
typedef struct node * PNODE;

int main ( )
{
    int num=1;
    PNODE head;

    head = (PNODE)malloc( sizeof(NODE) );
    head->next = NULL;
    head->data = -1;

    while ( num!=0 )
    {
        scanf("%d", &num);
        if ( num!=0 )
            sortlist( head, num);
    }
    outlist( head );
    return 0;
}

outlist( PNODE head )
{
    PNODE p;
    p = head->next;
    while ( p != NULL )
    {
        printf("%d\n", p->data);
        p = p->next;
    }
}

/* This is an example for list. Please programme your code like it.
sortlist( PNODE h, int num )
{
    PNODE p;
    p = (PNODE)malloc( sizeof(NODE) );
    p->data = num;
    p->next = h->next;
```

```
        h->next = p;
    }
    */

/* PRESET CODE END - NEVER TOUCH CODE ABOVE */

void sortlist(PNODE h, int num)
{
    // num 升序建立带有表头结点的链表
    PNODE p, t, t1;
    h->data=-999;
    p = (PNODE)malloc( sizeof(NODE) );
    p->data=num;
    p->next=NULL;
    if(h->next==NULL)
        h->next=p;
    else
    {
        t1=h;
        t=h->next;
        while(t->data<p->data&& t->next)
        {
            t1=t;
            t=t->next;
        }
        if(t->next==NULL&&t->data<p->data)
        {
            t->next=p;
        }
        else if(t->next==NULL&&t->data>p->data)
        {
            p->next=t1->next;
            t1->next=p;
        }
        else if(t->data==p->data)
        {}
        else
        {
            p->next=t1->next;
            t1->next=p;
        }
    }
}
```

45 子串反序--递归

请编写一个递归函数 `reverse(char str[], int start, int end)`，该函数的功能是将串 `str` 中下标从 `start` 开始到 `end` 结束的字符颠倒顺序。假设 `start` 和 `end` 都在合理的取值范围。

例如：

执行前：`str[]="0123456"`；`start=1`；`end=4`

执行后：`str[]="0432156"`

要求在该函数中不使用新的数组，没有循环。

	测试输入	期待的输出
测试用例 1	1. 0123456↵ 2. 1 4↵	1. 0432156↵

```
/* PRESET CODE BEGIN - NEVER TOUCH CODE BELOW */
#include <stdio.h>
int main( )
{
    char str[100];
    int start, end;
    gets(str);
    scanf("%d%d", &start, &end);
    reverse( str, start, end );
    printf("%s\n", str);
    return 0;
}
/* PRESET CODE END - NEVER TOUCH CODE ABOVE */
void reverse( char str[], int start, int end )
{
    int i=start, j=end;
    char temporary;
    for(; i<=j; i++, j--)
    {
        temporary=str[i];
        str[i]=str[j];
        str[j]=temporary;
    }
}
```